- A string representing the offset from UTC in hours and minutes in the form "+HH:MM" or "-HH:MM".
- An integer representing the offset from UTC in seconds.
- A single letter representing military time zones, as specified here.[1]
- A custom object that responds to the methods local_to_utc and utc_to_local with logic that performs the appropriate transition. (Time.new and Time.now can't take an object like this, but other time methods can.)

The next most common method used to create a new Time instance is Time.parse. The parse method is not available by default. To use Time.parse you must call require "time" somewhere in your code before use.

The Time.parse method takes a string and converts it to a Time object based on Ruby's best guess as to the underlying format. Missing parts of the time are set to 0, and missing parts of the date will be filled in based on the current date. If you want to use a different date as the baseline, you can pass in a date as a second positional argument.

You have some leeway in how you pass in the string and Ruby will try to do the right thing. Here are some examples:

```
require "time"
Time.parse("2023-2-10")                                  # => 2023-02-10 00:00:00
                                                         # .. -0600
Time.parse("2023-10-2")                                  # => 2023-10-02 00:00:00
                                                         # .. -0500
Time.parse("2023-2-28")                                  # => 2023-02-28 00:00:00
                                                         # .. -0600
Time.parse("1:00")                                       # => 2023-02-26 01:00:00
                                                         # .. -0600
Time.parse("February 26, 2023, 3:00 America/Chicago") # => 2023-02-26 03:00:00
                                                         # .. -0600
```

Time takes an optional block that accepts the year of the parsed time, and allows you to return a different year. This functionality is specifically there to allow you to manage two-digit year formats.

If you are using strings for dates as specified by HTTP protocol, you can use Time.httpdate to get the current date and time in the format used by HTTP requests. The format is day-of-week, DD month-name CCYY hh:mm:ss GMT. As I write this that corresponds to Sun, 26 Feb 2023 20:26:12 GMT. Similarly, you can use Time.rfc2822 to parse the very similar date format specified by that RFC.

If the unspecified nature of Time.parse bothers you, you can use Time.strptime (short for "string parse time"). The first argument to strptime is still a string representation of a time. The second argument is a format string, and the third argument is an optional date that fills in any missing date information. Like parse, an optional block can be used to manage two-digit years.

The format string uses the same format characters as strftime, which are shown in Table 14, directives, on page 498. The strptime method uses the format string to interpret the string and covert it to a Time. To use Time.strptime you must have called require time, as the strptime method is not available by default. Some examples:

1. https://en.wikipedia.org/wiki/List_of_military_time_zones